MapReduce: Simplified Data Processing on Large Clusters

Dean & Ghemawat, OSDI 2004

Kunal Juvvala

October 26, 2026

The Problem MapReduce Solves

Many custom data jobs

Same distributed plumbing

Simple logic, hard execution



The Abstraction

map(k, v) → list(k2, v2)

reduce(k2, list(v2)) → list(v2)

Runtime owns partitioning, scheduling, shuffle, failures



Why the Runtime Scales

Re-execution

Dead workers trigger task rescheduling

Data locality

Move compute to input blocks

Combiners + partitioning

Shrink shuffle; route by hash(key) mod R

Backup tasks

Duplicate stragglers near the end

HW4: Structure vs. Guarantees

- Split → map → reduce
- Shared storage ≈ GFS
- verify.py checks output
- Missing failure handling
- No straggler mitigation

Distributed Systems Threads Beyond HW4

Fan-out / fan-in

Goroutines, channels, and worker pools mirror map and reduce stages

Less shared-state locking

Reducers own disjoint key partitions instead of fighting over one map

Observability first

Master status and counters anticipate dashboards and alerting

Horizontal scaling

Stateless workers scale out the same way map tasks do

MapReduce scales by hiding distribution work—not by making network, coordination, or serial overhead disappear.